

Szakképesítés megnevezése: Szoftverfejlesztő és -tesztelő
Azonosító száma: 5 0613 12 03

VIZSGAREMEK

Luminé Iskolai Adminisztrációs Rendszer

Készítették:

Gaál Levente	Osztály: 13.C	Oktatási azonosító: 72824052514
Kőszegi Bence	Osztály: 13.C	Oktatási azonosító: 72751848289
Kurtucz F. Krisztián	Osztály: 13.C	Oktatási azonosító: 72736021625

Békéscsaba, 2025/2026

Tartalomjegyzék

Tartalomjegyzék.....	2
Összefoglaló.....	4
1. Bevezető.....	6
1.1 A probléma ismertetése	6
1.2 A megoldás, Luminé rendszer	6
1.3 Célközönség és szerepkörök.....	7
2. Témaválasztás indoklása.....	7
2.1 Személyes motiváció	7
2.2 Technológiai kihívások.....	7
2.3 Üzleti érték	8
2.4 Piaci helyzet.....	8
3. Fejlesztői dokumentáció.....	9
3.1 Funkcionális követelmények	9
3.2 Nem-funkcionális követelmények.....	9
3.3 Rendszerterv és architektúra.....	10
3.3.1 Háromrétegű architektúra	10
3.3.2 Technológiai stack	11
3.3.3 Projekt mappastruktúra	11
3.4 Adatbázis leírása.....	12
3.4.1 Adatbázis típus és indoklás	12
3.4.1.1 Technológiai előnyök az SQL-lel szemben	10
3.4.1.2 Stratégiai denormalizáció	11
3.4.1.3 Adatbázis biztonsági megfontolások.....	11
3.4.2 Firestore kollekciók	14
3.4.3 ER Diagram - Entitás-kapcsolat modell	16
3.4.4 Indexek és optimalizáció.....	24

3.4.5 Cloudinary - Képadatbázis.....	25
3.5 API dokumentáció	26
3.5.1 API architektúra	26
3.5.2 API végpontok	26
3.5.3 HTTP státuszkódok.....	28
3.6 Biztonsági megvalósítás	28
3.6.1 Autentikáció és autorizáció	29
3.6.2 RBAC mátrix	29
3.6.3 Biztonsági intézkedések összefoglalója	30
3.7 Tesztelés	31
3.7.1 Tesztelési stratégia	31
3.7.2 Tesztelési eredmények	31
3.7.3 Teljesítmény metrikák	32
3.7.4 Főbb tesztesetek	32
3.8 Telepítés és üzemeltetés	34
3.8.1 Rendszerkövetelmények	34
3.8.2 Telepítési lépések.....	34
4. Felhasználói dokumentáció	39
4.1 Rendszerkövetelmények.....	39
4.2 Bejelentkezés	39
Demo bejelentkezési adatok	39
4.3 Diák felhasználói útmutató.....	39
4.4 Tanár felhasználói útmutató	40
4.5 Osztályfőnök felhasználói útmutató	40
4.6 Szülő felhasználói útmutató.....	41
4.7 Admin felhasználói útmutató.....	41
4.8 Igazgató felhasználói útmutató.....	41

4.9 Hibaelhárítás - GYIK.....	41
5. Összegzés	42
5.1 Projekt eredményei	42
5.2 Fejlesztési tapasztalatok	42
Pozitív tapasztalatok	42
Kihívások és megoldások	42
5.3 Jövőbeli fejlesztési lehetőségek.....	43
6. Irodalomjegyzék.....	44
6.1 Könyvek és publikációk	43
6.2 Online dokumentációk.....	44
6.3 Webes és technikai források	44

Ábrajegyzék

1. ábra: Lighthouse teljesítmény riport (FCP, LCP, Accessibility)	
2. ábra: Firebase Console Web app regisztrálás és firebaseConfig beállítása	
3. ábra: Firebase Admin SDK privát kulcs letöltése	
4. ábra: Firebase Authentication Email/Password beállítás	
5. ábra: Firestore adatbázis létrehozása és beállítása	
6. ábra: Firestore biztonsági szabályok (Security Rules) beállítása	
7. ábra: Cloudinary Dashboard Cloud Name és Upload Preset konfiguráció.....	
8. ábra: Cloudinary Upload Preset létrehozása (Unsigned mód)	

Összefoglaló

A Luminé egy modern, webalapú iskolai adminisztrációs rendszer, amelyet a Békéscsabai SZC Nemes Tihamér Technikum és Kollégium számára fejlesztettünk. A rendszer Next.js 14 és Firebase technológiákra épül, teljes körű megoldást nyújtva az oktatási intézmények mindennapi adminisztrációs feladataihoz.

A rendszer hét különböző felhasználói szerepkört támogat, és funkcióinak köre felöleli a jegykezelést, órarend-kezelést, hiányzásnyilvántartást, házi feladatok kezelését, valós idejű kommunikációt, szülői hozzáférést, QR-kódos beléptetést és iskolai rádió zenekérési rendszert.

Jellemző	Érték
Projekt neve	Luminé Iskolai Adminisztrációs Rendszer
Készítette	Gaál Levente, Kőszegi Bence, Kurtucz F. Krisztián
Vizsga éve	2025/2026
Intézmény	Békéscsabai SZC Nemes Tihamér Technikum és Kollégium
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS
Backend	Next.js API Routes (RESTful)
Adatbázis	Firebase Firestore (NoSQL)
Autentikáció	Firebase Authentication (JWT)
Hosting	Vercel
Tesztelés	Vitest (unit), Playwright (E2E)
Szerepkörök száma	7 (admin, igazgató, osztályfőnök, tanár, diák, DJ, szülő)
Unit tesztek	15/15 sikeres
E2E tesztek	23/23 sikeres

1. Bevezető

1.1 A probléma ismertetése

Az oktatási intézmények adminisztrációs folyamatai hagyományosan papíralapúak vagy elavult szoftverekkel támogatottak. A tanárok, diákok, szülők és az iskolavezetés közötti kommunikáció gyakran lassú, nem hatékony, és az információk szétszórtnak, különböző csatornákon keresztül érhetők el.

A jegyek, hiányzások, házi feladatok nyilvántartása manuális munkát igényel, amely időigényes és hibalehetőségeket rejt magában. Az iskolák számára szükség volt egy modern, integrált rendszerre.

Fő problémák:

- Szétszórtnak kommunikáció: különböző csatornák, lassú információáramlás
- Nem hatékony jegykezelés: késői visszajelzések
- Elavult design: nem a modern generációk használatát támogatja

1.2 A megoldás, Luminé rendszer

A Luminé egy modern, webalapú iskolai adminisztrációs rendszer, amely Next.js 14 és Firebase technológiákra épül. Teljes körű digitális megoldást nyújt az oktatási intézmény mindennapi működéséhez.

Főbb funkciói:

- Jegykezelés automatikus átlagszámítással és statisztikákkal
- Órendezés kezelése heti és napi nézetben, helyettesítések jelölésével
- Hiányzások nyilvántartása és online igazolási rendszer
- Házi feladatok kiadása, online beadása és értékelése
- Viselkedési bejegyzések (dicséret, figyelmeztetés) három szinten
- Valós idejű kommunikáció (chat, üzenőfal)
- Szülői hozzáférés gyermek adataihoz
- QR kódos be- és kiléptetés
- Iskolai rádió zenekérés rendszer

1.3 Célközönség és szerepkörök

A rendszer hét különböző felhasználói szerepkört támogat, mindegyik saját dashboarddal és funkcionalitással:

Szerepkör	Magyar neve	Fő feladatok
admin	Adminisztrátor	Teljes rendszeradminisztráció, felhasználók kezelése, órarend szerkesztés
principal	Igazgató	Intézményi áttekintés, statisztikák, igazgatói szintű viselkedési bejegyzések
homeroom_teacher	Osztályfőnök	Osztálykezelés, igazolások jóváhagyása, magatartás/szorgalom jegyek
teacher	Tanár	Jegyek rögzítése, jelenlét kezelés, házi feladatok kiadása, szaktanári bejegyzések
student	Diák	Saját adatok megtekintése, házi feladatok beadása, chat, QR kód, zenekérés
dj	DJ	Diák jogkörök + zenekérések kezelése az iskolai rádióhoz
parent	Szülő	Gyermek adatainak követése (jegyek, hiányzások, órarend), kommunikáció

2. Témaválasztás indoklása

2.1 Személyes motiváció

Az oktatási rendszer digitalizálása napjaink egyik legfontosabb kihívása. A COVID-19 járvány rávilágított arra, mennyire elengedhetetlen a modern, digitális eszközök alkalmazása az oktatásban. A Luminé projekt célja, hogy egy átfogó megoldást nyújtson, amely nem csak válsághelyzetekben, hanem a mindennapi iskolai működés során is hatékonyan támogat.

2.2 Technológiai kihívások

A projekt megvalósítása során az alábbi főbb technológiai kihívásokkal kellett megbirkózni:

- Skálázhatóság több száz felhasználó egyidejű kiszolgálása
- Valós idejű adatszinkronizáció chat és órarend-változások azonnali megjelenítése
- Szerepkör alapú hozzáférés-kezelés 7 különböző jogosultsági szint
- Reszponzív design és optimális megjelenés minden eszközön

2.3 Üzleti érték

A Luminé bevezetése kézzelfogható előnyöket hoz az iskoláknak. Nem csak kényelmesebb lesz a működés, hanem időt, pénzt és energiát is spórol.

Kategória	Várható eredmény
Időmegtakarítás	Tanároknál 30-40%-os adminisztrációs idő csökkentés
Papírmentes működés	Nyomtatási és papírköltségek megszüntetése
Átláthatóság	Minden adat egy helyen, valós idejű statisztikák
Szülők bevonása	Azonnali hozzáférés gyermek adataihoz, kommunikáció
Kommunikáció	Egységes platform, értesítések, chat

2.4 Piaci helyzet

A magyar oktatási piacon több elektronikus napló rendszer is elérhető (pl. KRÉTA), azonban ezek többsége elavult technológiával készült, nem felhasználóbarát, és drága licencdíjakat számít fel. A Luminé ezzel szemben modern technológiákra épül, nyílt forráskódú és könnyen testre szabható.

3. Fejlesztői dokumentáció

3.1 Funkcionális követelmények

A rendszer funkcionális követelményeit az alábbi táblázat foglalja össze. A követelmények a szoftver tervezett viselkedését írják le.

Azonosító	Leírás	Prioritás
FR-01	A rendszer támogatja a bejelentkezést email/jelszó alapján	Kötelező
FR-02	Hét különböző felhasználói szerepkör kezelése	Kötelező
FR-03	Szerepkör alapú hozzáférés kezelés (RBAC)	Kötelező
FR-04	Tanárok jegyeket rögzíthetnek (1-5) tantárgyanként	Kötelező
FR-05	Automatikus tantárgyi átlagszámítás	Kötelező
FR-06	Órarend megtekintése heti és napi nézetben	Kötelező
FR-07	Tanárok jelölhetnek helyettesítéseket, elmaradó órákat	Kötelező
FR-08	Hiányzások rögzítése és igazolási rendszer	Kötelező
FR-09	Házi feladatok kiadása, beadása és értékelése	Kötelező
FR-10	Valós idejű chat kommunikáció	Kötelező
FR-11	Viselkedési bejegyzések három szinten	Kötelező
FR-12	Szülői hozzáférés gyermek adataihoz	Kötelező
FR-13	QR kódos be-/kiléptetés	Ajánlott
FR-14	Iskolai rádió zenekérés rendszer	Opcionális
FR-15	Profil kezelés (kép, elérhetőség)	Ajánlott

3.2 Nem-funkcionális követelmények

A nem-funkcionális követelmények a rendszer minőségi jellemzőit határozzák meg.

Kategória	Követelmény	Cél érték
Teljesítmény	Oldal betöltési idő	< 3 s
Teljesítmény	API válaszütem	< 500 ms átlag
Teljesítmény	Egyidejű felhasználók	500+
Teljesítmény	First Contentful Paint	0.8 s
Teljesítmény	Largest Contentful Paint	1.4 s
Biztonság	Autentikáció minden API végponton	Kötelező
Biztonság	Firestore Security Rules	Kötelező
Biztonság	HTTPS titkosítás	TLS 1.3
Biztonság	GDPR megfelelés	Kötelező
Megbízhatóság	Uptime	≥ 99.5%
Megbízhatóság	Offline cache	Service Worker
Használhatóság	Reszponzív design	Mobil + Asztali
Használhatóság	Akadálymentesség	WCAG 2.1 AA
Karbantarthatóság	Unit teszt lefedettség	≥ 90%
Skálázhatóság	Adatbázis auto-skálázás	Firebase Firestore

3.3 Rendszerterv és architektúra

3.3.1 Háromrétegű architektúra

A Luminé rendszer háromrétegű architektúrát követ, amely tisztán elkülöníti a megjelenítési, alkalmazás- és adatrétegeket:

Prezentációs réteg - Next.js Frontend (React, Tailwind CSS)

Alkalmazás réteg - Next.js API Routes (RESTful, üzleti logika, RBAC)

Adat réteg - Firebase Firestore + Firebase Authentication + Cloudinary kép tárolás

3.3.2 Technológiai stack

Réteg	Technológia	Verzió	Felhasználás
Frontend	Next.js	14.x	React framework, App Router, SSR/SSG
Frontend	React	18.x	UI komponens könyvtár, Hooks
Frontend	TypeScript	5.x	Típusbiztos JavaScript szuperset
Frontend	Tailwind CSS	3.x	Utility-first CSS framework
Frontend	Radix UI	latest	Akadálymentes headless UI komponensek
Frontend	Lucide React	latest	Ikonkönyvtár
Backend	Next.js API Routes	14.x	RESTful API végpontok
Backend	Firebase Admin SDK	12.x	Szerver oldali Firebase műveletek
Backend	Node.js	18+	JavaScript runtime
Adatbázis	Firebase Firestore	v9+	NoSQL dokumentum adatbázis
Auth	Firebase Authentication	v9+	JWT token alapú hitelesítés
Tesztelés	Vitest	latest	Unit tesztelési framework
Tesztelés	Playwright	latest	End-to-end tesztelés
Devtools	ESLint	latest	Kód minőség ellenőrzés

3.3.3 Projekt mappastruktúra

A projekt az alábbi könyvtárstruktúrát követi:

```
lumine-app/  
├─ e2e/                # End-to-end tesztek (Playwright)  
├─ public/             # Statikus fájlok (logo, sw.js)  
├─ scripts/           # Segédscriptek (test-adatbazis.js)  
└─ src/  
    ├─ app/            # Next.js App Router  
    │   └─ api/         # API végpontok  
    │       └─ academic/ # Jegyek, hiányzás, házi feladat, órarend  
    │       └─ admin/    # Admin műveletek  
    │       └─ auth/     # Autentikáció  
    │       └─ behavior/  # Viselkedési bejegyzések  
    │       └─ communication/ # Chat  
    │       └─ music/     # Iskolai rádió  
    │       └─ parent/    # Szülői funkciók  
    │       └─ users/     # Felhasználók  
    └─ dashboard/       # Dashboard oldalak  
├─ contexts/           # React Context-ek  
├─ lib/                # Utility (auth, permissions, firebase)  
├─ shared/             # Megosztott komponensek és típusok  
└─ tests/              # Unit tesztek
```

3.4 Adatbázis leírása

3.4.1 Adatbázis típus és indoklás

A Luminé rendszer adatbázisának tervezésekor a legfontosabb szempont a valós idejű működés, a skálázhatóság és a Firebase ökoszisztémával való szoros integráció volt. A választott DBMS a Google Firebase Firestore, egy felhőalapú NoSQL dokumentum-adatbázis. Az adatstruktúra kialakítása során figyelembe vettük a NoSQL sajátosságait: a hagyományos relációs normalizáció elvei helyett a lekérdezési mintákhoz igazodó, denormalizált dokumentumszerkezetet alkalmaztunk. Az entitások közötti kapcsolatokat referencia mezők (idegen kulcsok, FK) jelölik, az összetett kapcsolatokat pedig az ER modell szemlélteti.

3.4.1.1 Technológiai előnyök az SQL-lel szemben

A Luminé rendszer Firebase Firestore NoSQL dokumentum adatbázist alkalmaz. A tervezés korai fázisában megvizsgáltuk a hagyományos SQL alapú megoldásokat (pl. PostgreSQL, MySQL) is, azonban a projekt követelményei valós idejű szinkronizáció, automatikus skálázás, Firebase Auth integráció a Firestore mellett szóltak. Az alábbiakban a választást indokló szempontok, illetve az SQL-lel való összehasonlítás látható:

Szempont	Indoklás
Valós idejű szinkronizáció	Firestore real-time listener, chat, órarend frissítések
Automatikus skálázás	Nincs szükség manuális szerver menedzsmentre
Beépített biztonság	Firestore Security Rules az adatbázis szintű védelemhez
Firebase integráció	Zökkenőmentes Firebase Auth + Firestore együttműködés
Offline támogatás	Beépített offline cache és szinkronizáció
Ingyenes tier	Fejlesztéshez és kis léptékű használathoz ingyenes

Firestore vs. SQL összehasonlítás a projekt szempontjából:

Szempont	Firestore (NoSQL)	SQL (pl. PostgreSQL)
Valós idejű frissítés	Beépített real-time listener	Websocket/polling szükséges
Skálázhatóság	Automatikus, szerver nélkül	Manuális szerver konfiguráció
Auth integráció	Firebase Auth natív támogatás	Külön megvalósítás szükséges
Adatstruktúra rugalmassága	Sémamentes, role-függő mezők	Kötött séma, migráció szükséges
JOIN műveletek	Nem támogatott, denormalizációval megkerülhető	Teljes körű JOIN támogatás

3.4.1.2 Stratégiai denormalizáció

A Firestore NoSQL adatbázis természetéből adódóan nem támogat JOIN műveleteket. Ennek kompenzálására a Luminé rendszer stratégiai denormalizációt alkalmaz: a leggyakrabban szükséges adatokat (pl. diák neve, osztálya; tanár neve) redundánsan tárolja több kollekcióban is. Ez a megközelítés csökkenti a lekérdezések számát, javítja a válaszütemet, és lehetővé teszi az összetett adatok egyetlen olvasással történő visszanyerését.

Denormalizált mező	Érintett kollekciók	Indok
studentName	absences, grades, behavior_records, excuses, homework-submissions	Elkerüli a users kollekció külön lekérdezését
teacherName	lessons, homework, grades	Gyors megjelenítés külön join nélkül
studentClass	grades, behavior_records, excuses	Osztályszintű szűrés egyetlen lekérdezéssel

3.4.1.3 Adatbázis biztonsági megfontolások

Az adatbázis tervezésekor kiemelten figyeltünk a biztonsági szempontokra. A Firestore biztonsági modellje két szinten érvényesül: szerver oldalon a Firebase Admin SDK-n keresztül, illetve adatbázis szinten a Firestore Security Rules szabályrendszer által. Ez azt jelenti, hogy még ha valaki megkerülné is az API réteget, a közvetlen adatbázis-hozzáférés is jogosultság-ellenőrzés alá esik.

Biztonsági elem	Megvalósítás
Firestore Security Rules	Adatbázis szintű jogosultság-ellenőrzés: minden kollekció csak az arra jogosult szerepköri token alapján olvasható/írható. Közvetlen kliens oldali hozzáférés is védett.
UID alapú adatizoláció	Minden felhasználó csak a saját UID-jéhez kötött dokumentumokat érheti el (pl. diák csak saját jegyeit, szülő csak saját gyermekének adatait látja).
Érzékeny mezők védelme	Jelszavak soha nem kerülnek Firestore-ba azokat kizárólag a Firebase Authentication kezeli. A profileImage, email és telefonszám mezők csak az érintett felhasználó vagy admin számára olvashatók.
Írási jogosultságok korlátozása	A grades, behavior_records és lessons kollekciók írása kizárólag tanár, osztályfőnök vagy admin role-lal lehetséges. A Security Rules szerver oldalon is érvényesítik a RBAC logikát.
Audit trail	Minden műveletvégző kollekció tartalmaz createdAt és recordedBy/teacherId mezőt, amely nyomon követhetővé teszi, ki és mikor módosította az adatot.

3.4.2 Firestore kollekciók

Az adatbázis 15 fő kollekciót tartalmaz:

Kollekció neve	Tartalom	Kapcsolat
absences	Egyéni hiányzás bejegyzések (tanár rögzíti óránként)	users (studentId, teacherId), lessons
access	QR-kódos be- és kilépési naplók (entryTime, exitTime)	users (studentId)
attendance	Órai jelenléti ívek - diákok tömbje (present, excused)	lessons, users (teacherId)
behavior_records	Viselkedési bejegyzések: dicséret és figyelmeztetés	users (studentId, recordedBy)
chatMessages	Valós idejű chat üzenetek (message, sender)	users (sender)
classes	Osztályok nyilvántartása (name: "9.A" stb.)	Referencia tábla
excuses	Igazolási kérelmek szülő nyújtja be absenceIds-sel	users (studentId), absences
grades	Jegyek: tantárgyi és magatartás/szorgalom (isBehaviorGrade)	users (studentId, teacherName)
homework	Házi feladatok (status: active, attachments tömb)	users (teacherId), lessons
homework-submissions	Beadott házi feladatok - Cloudinary mellékletekkel	homework, users (studentId)
lessons	Órarend bejegyzések (day, startTime, room, status)	users (userId=teacherId)
musicRequests	Rádió zenekérések: YouTube URL, platform, thumbnail	users (userId)
parent_children	Szülő-gyermek összekapcsolások (childStudentId, verified)	users (parentId, childId)
schedule-changes	Órarend változások: changeType, eredeti és új adatok	lessons (timeSlot alapján)
users	Felhasználók role-függő mezőkkel (admin, tanár, diák stb.)	Alap entitás

3.4.3 ER Diagram - Entitás-kapcsolat modell

absences kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
className	string	Osztály neve (pl. "9.A")
createdAt	timestamp (ISO)	Létrehozás ideje
date	string (YYYY-MM-DD)	Hiányzás dátuma
excused	boolean	Igazolt-e a hiányzás
excusedAt	timestamp (ISO)?	Igazolás időpontja
excusedBy	string?	Igazoló neve (pl. "Osztalyfonok1")
lessonId	string	Óra azonosító (pl. "Kedd_10:45_9.A")
startTime	string (HH:MM)	Óra kezdési ideje
studentId	string (FK→users)	Diák Firebase Auth UID
studentName	string	Diák neve (denormalizált)
subject	string	Tantárgy neve
teacherId	string (FK→users)	Tanár Firebase Auth UID
topic	string	Óra témája (lehet üres)

access kollekció

Mező	Típus	Leírás
id (🔗)	string	Format: studentId_YYYY-MM-DD
date	string (YYYY-MM-DD)	Belépés dátuma
entryTime	timestamp (ISO)	Belépés pontos időpontja
exitTime	timestamp (ISO)	Kilépés pontos időpontja
studentId	string (FK→users)	Diák Firebase Auth UID

attendance kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
className	string	Osztály neve
createdAt	timestamp (ISO)	Rögzítés ideje
date	string (YYYY-MM-DD)	Óra dátuma
lessonId	string	Óra azonosító (pl. "Péntek_7:45_9.B")
startTime	string (HH:MM)	Óra kezdési ideje
students	array of object	Diákok jelenléti adatai (ld. lent)
students[].excused	boolean	Igazolt hiányzás-e
students[].present	boolean	Jelen volt-e
students[].studentId	string (FK→users)	Diák UID
students[].studentName	string	Diák neve (denormalizált)

behavior_records kollekció

Ez a kollekció tárolja az összes viselkedési bejegyzést: szaktanári, osztályfőnöki és igazgatói szintű dicséreteket és figyelmeztetéseket egyaránt.

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
createdAt	timestamp (ISO)	Bejegyzés ideje
description	string	Részletes leírás
level	string (enum)	"szaktanari" "osztalyfonoki" "igazgatoi"
parentNotified	boolean	Értesítve lett-e a szülő
reason	string	Ok/indoklás (lehet üres)
recordedBy	string (FK→users)	Rögzítő UID
recordedByName	string	Rögzítő neve (denormalizált)
studentClass	string	Diák osztálya (denormalizált)
studentId	string (FK→users)	Diák UID
studentName	string	Diák neve (denormalizált)
type	string (enum)	"dicseret" "figyelmezetes"

chatMessages kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
createdAt	timestamp (ISO)	Üzenet küldésének ideje
message	string	Üzenet szövege (max. 500 karakter)
sender	string	Küldő neve (denormalizált, nem UID!)

classes kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
createdAt	timestamp (ISO)	Létrehozás ideje
name	string	Osztály neve (pl. "9.A")

excuses kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
absenceIds	string[]	Igazolandó hiányzások ID-i (FK→absences)
description	string	Igazolás leírása / ok
excuseType	string (enum)	"betegség" "egyéb" stb.
reviewedAt	timestamp (ISO)?	Elbírálás időpontja
reviewedBy	string?	Elbíráló neve (pl. "Osztályfonok2")
status	string (enum)	"pending" "approved" "rejected"
studentClass	string	Diák osztálya (denormalizált)
studentId	string (FK→users)	Diák UID
studentName	string	Diák neve (denormalizált)
submittedAt	timestamp (ISO)	Benyújtás időpontja
submittedBy	string	Benyújtó neve (szülő neve)

grades kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firestore auto ID
createdAt	timestamp (ISO)	Rögzítés ideje
date	timestamp (ISO)	Jegy dátuma
description	string	Megjegyzés / leírás
grade	number (1-5)	Jegy értéke
isBehaviorGrade	boolean	Magatartás/szorgalom jegy-e
studentClass	string	Diák osztálya (denormalizált)
studentId	string (FK→users)	Diák UID (lehet üres ha névvel keresi)
studentName	string	Diák neve (denormalizált)
subject	string	Tantárgy neve
teacherName	string	Tanár neve (denormalizált)
title	string	Jegy címe (pl. "Felelet", "Magatartás - 06. hónap")
type	string	Jegy típusa (pl. "2026-06" havi magatartásnál)

homework kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firestore auto ID
attachments	string[] (üres tömb)	Cloudinary URL-ek (opcionális)
className	string	Osztály neve (denormalizált)
createdAt	timestamp (ISO)	Létrehozás ideje
description	string	Feladat leírása
dueDate	string (YYYY-MM-DD)	Beadási határidő
lessonId	string (FK→lessons)	Óra azonosítója
status	string (enum)	"active" "closed"
subject	string	Tantárgy neve (denormalizált)
teacherId	string (FK→users)	Tanár UID
teacherName	string	Tanár neve (denormalizált)
title	string	Feladat címe

homework-submissions kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
attachments	string[]	Cloudinary URL-ek (pl. res.cloudinary.com/dbi2rvspr/...)
content	string	Szöveges beadás tartalma
evaluated	boolean	Értékelte-e már a tanár
evaluatedAt	timestamp (ISO)?	Értékelés időpontja
feedback	string	Tanár visszajelzése (lehet üres)
grade	null string	Értékelés (null ha még nem értékelt)
homeworkId	string (FK→homework)	Házi feladat ID
status	string (enum)	"incomplete" "complete"
studentId	string (FK→users)	Diák UID
studentName	string	Diák neve (denormalizált)
submittedAt	timestamp (ISO)	Beadás időpontja

lessons kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
className	string	Osztály neve
createdAt	timestamp (ISO)	Létrehozás ideje
day	string	"Hétfő" "Kedd" ... "Péntek"
room	string	Teremszám (pl. "111")
startTime	string (HH:MM)	Óra kezdési ideje
status	string (enum)	"normal" "helyettesítés" "elmarad"
subject	string	Tantárgy neve
teacherName	string	Tanár neve (denormalizált)
userId	string (FK→users)	Tanár UID

musicRequests kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
createdAt	timestamp (ISO)	Kérés beküldésének ideje
platform	string	"youtube" egyéb platform
thumbnail	string (URL)	Videó bélyegképe (YouTube thumbnail URL)
title	string	Videó/dal neve
url	string (URL)	YouTube vagy más lejátszási link
userClass	string	Kérő osztálya (lehet "N/A" ha nincs)
userId	string	Kérő UID ("anonymous" ha névtelen)
userName	string	Kérő neve (pl. "Névtelen diák")

parent_children kollekció

Mező	Típus	Leírás
id (🔗)	string	Format: parentId__childId
childClass	string	Gyermek osztálya (denormalizált)
childId	string (FK→users)	Gyermek UID
childName	string	Gyermek neve (denormalizált)
childStudentId	string	OM azonosító szám
linkedAt	timestamp (ISO)	Összekapcsolás időpontja
parentId	string (FK→users)	Szülő UID
relationship	string (enum)	"anya" "apa" "gyám" "egyéb"
verified	boolean	Admin által jóváhagyott-e

schedule-changes kollekció

Mező	Típus	Leírás
id (🔗)	string (auto)	Firebase auto ID
changeType	string (enum)	"cancelled" "substitution" "room_change" stb.
createdAt	timestamp (ISO)	Rögzítés ideje
date	string (YYYY-MM-DD)	Érintett óra dátuma
newClass	string	Új osztály (ha változott)
newRoom	string	Új terem (üres ha törlés)
newSubject	string	Új tantárgy (üres ha törlés)
newTeacher	string	Helyettesítő tanár neve (üres ha törlés)
originalClass	string	Eredeti osztály neve
originalTeacher	string	Eredeti tanár neve
teacherId	string	Érintett tanár/osztály azonosítója
timeSlot	string (HH:MM)	Érintett óra időpontja

users kollekció (role-függő mezők)

A users kollekció minden felhasználónál tárolja az alap mezőket, de a role értékétől függően további role-specifikus mezők is jelen lehetnek.

Mező	Típus	Szerepkörök
id (🔗)	string (auto)	Minden szerepkör
address	string?	Minden szerepkör (opcionális)
createdAt	timestamp (ISO)	Minden szerepkör
email	string	Minden szerepkör
fullName	string	Minden szerepkör
name	string	Minden szerepkör (megjelenítési név)
phone	string?	Minden szerepkör (opcionális)
profileImage	string? (Cloudinary URL)	Minden szerepkör (opcionális)
role	string (enum)	Minden szerepkör
updatedAt	timestamp (ISO)	Minden szerepkör
class	string	student, dj
classes	string[]	homeroom_teacher
subjects	string[]	teacher, homeroom_teacher

Táblák közötti kapcsolatok

Kapcsolat	Mező (FK)	Kard.	Megjegyzés
users → absences	absences.studentId, teacherId	1:N	Egy diáknak több hiányzása
users → access	access.studentId	1:N	QR belépési napló
lessons → attendance	attendance.lessonId	1:N	Órai jelenléti ív
users → behavior_records	records.studentId, recordedBy	1:N	Összes szintű bejegyzés
absences → excuses	excuses.absenceIds[]	N:M	Egy igazolás több hiányzást fed
users → grades	grades.studentId	1:N	Tantárgyi és magatartás jegy
users → homework	homework.teacherId	1:N	Tanár adja fel
lessons → homework	homework.lessonId	1:N	Órai házi feladat
homework → hw-submissions	submissions.homeworkId	1:N	Egy feladathoz több beadás
users → hw-submissions	submissions.studentId	1:N	Diák beadása
users → lessons	lessons.userId (tanár)	1:N	Tanár óráinak listája
users → musicRequests	musicRequests.userId	1:N	Diák zenekérése
users → parent_children	parent_children.parentId/childId	N:M	Szülő-gyermek kapcsolat
classes → users	users.class (string ref)	1:N	Osztály → diákok
lessons → schedule-changes	schedule-changes.timeSlot+date	1:N	Óra módosítása/törlése

3.4.4 Indexek és optimalizáció

A Firebase Firestore minden egyes mezőt automatikusan indexel egyszerű (single-field) lekérdezésekhez. Az összetett (composite) indexeket kézzel kell létrehozni a Firebase Console-ban a projekt Indexes fülén jelenleg manuálisan definiált composite index nincs konfigurálva.

A lekérdezési minták alapján az alábbi composite indexek létrehozása javasolt a teljesítmény optimalizálásához:

Kollekció	Javasolt composite index mezők	Felhasználás	Prioritás
absences	studentId + date	Diák hiányzásai dátum szerint	Magas
absences	lessonId + date	Óra összes hiányzása	Magas
attendance	lessonId + date	Órai jelenléti ív lekérése	Magas
grades	studentId + subject + date	Diák jegyei tantárgyanként	Magas
grades	studentClass + isBehaviorGrade + date	Osztály magatartás jegyei	Közepes
homework	className + dueDate	Osztály házi feladatai határidő sz.	Magas
homework	teacherId + status	Tanár aktív feladatai	Közepes
homework-submissions	homeworkId + studentId	Beadás keresés	Magas
homework-submissions	studentId + evaluated	Értéketlen beadások	Közepes
lessons	userId + day	Tanár napi órarendje	Magas
lessons	className + day + startTime	Osztály órarendje sorrendben	Magas
schedule-changes	date + originalClass	Napi órarendváltozások osztálynak	Közepes
behavior_records	studentId + createdAt	Diák viselkedési bejegyzései	Közepes
excuses	studentId + status	Függőben lévő igazolások	Közepes
musicRequests	userId + createdAt	DJ zenekérések kezelése	Alacsony

Megjegyzés: A Firestore automatikusan generál index létrehozási linket, ha az alkalmazás kód olyan lekérdezést futtat, amelyhez nincs composite index. A fenti indexek létrehozásával a compound lekérdezések válaszüzeje jelentősen javítható.

3.4.5 Cloudinary - Képadatbázis

A Luminé rendszer Cloudinary felhőalapú médiatárolást alkalmaz profilképek és házi feladat mellékletek kezeléséhez. A valós adatbázisban látható URL-ek formátuma: `https://res.cloudinary.com/dbi2rvspr/image/upload/...` - ez megerősíti, hogy a rendszer aktívan használja a Cloudinary-t.

Paraméter	Érték
NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME	dbi2rvspr
NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET	secondstyle (unsigned)
CDN alap URL	https://res.cloudinary.com/dbi2rvspr/image/upload/
Támogatott formátumok	JPG, PNG, WEBP, GIF
Maximális fájl méret	10 MB

Feltöltési folyamat

A képfeltöltés unsigned upload preset segítségével, közvetlenül a böngészőből a Cloudinary API-ra történik:

```
const formData = new FormData()
formData.append("file", imageFile)
formData.append("upload_preset", "secondstyle")
const res = await
fetch("https://api.cloudinary.com/v1_1/dbi2rvspr/image/upload",
  {{ method: "POST", body: formData }})
const {{ secure_url }} = await res.json() // Mentés Firestore-ba
```

Felhasználás az adatbázisban

Cloudinary URL megjelenik	Firestore kollekció	Mező neve
Profilképek	users	profileImage (string)
Házi feladat beadás melléklet	homework-submissions	attachments (string[])
Házi feladat csatolmány (tanár)	homework	attachments (string[])

3.5 API dokumentáció

3.5.1 API architektúra

A Luminé RESTful API architektúrát alkalmaz, amelyet a Next.js 14 App Router API Routes valósít meg. Minden végpont autentikáció-ellenőrzést végez a Firebase JWT token alapján.

Jellemző	Megvalósítás
API stílus	RESTful (GET, POST, PUT, DELETE)
Autentikáció	Bearer token (Firebase JWT)
Autorizáció	Szerepkör ellenőrzés minden végponton
Adatformátum	JSON
Hibakezelés	HTTP státuszkódok + JSON hibaüzenet

3.5.2 API végpontok

Metódus	Végpont	Leírás	Jogosultság
POST	/api/auth/login	Bejelentkezés	Nyilvános
POST	/api/auth/logout	Kijelentkezés	Hitelesített
GET	/api/users	Felhasználók listája	admin
POST	/api/users	Új felhasználó létrehozása	admin
GET	/api/users/[id]	Felhasználó adatai	admin, saját
PUT	/api/users/[id]	Felhasználó módosítása	admin, saját
DELETE	/api/users/[id]	Felhasználó törlése	admin
GET	/api/academic/grades	Jegyek lekérése	teacher, student, parent
POST	/api/academic/grades	Jegy rögzítése	teacher, homeroom_teacher
DELETE	/api/academic/grades/[id]	Jegy törlése	admin, teacher
GET	/api/academic/attendance	Jelenléti ívek lekérése	teacher, student, parent
POST	/api/academic/attendance	Jelenléti ív rögzítése	teacher, homeroom_teacher
GET	/api/academic/homework	Házi feladatok lekérése	all
POST	/api/academic/homework	Házi feladat kiadása	teacher
PUT	/api/academic/homework/[id]	Házi feladat módosítása	teacher (saját)

Metódus	Végpont	Leírás	Jogosultság
DELETE	/api/academic/homework/[id]	Házi feladat törlése	teacher, admin
POST	/api/academic/homework/submit	Házi feladat beadása	student, dj
GET	/api/academic/lessons	Órarend lekérése	all
POST	/api/academic/lessons	Óra hozzáadása	admin
PUT	/api/academic/lessons/[id]	Óra módosítása	admin
DELETE	/api/academic/lessons/[id]	Óra törlése	admin
GET	/api/absences	Hiányzások lekérése	teacher, student, parent
POST	/api/absences	Hiányzás rögzítése	teacher
PUT	/api/absences/[id]	Hiányzás módosítása (igazolás)	homeroom_teacher
GET	/api/behavior	Viselkedési bejegyzések lekérése	teacher, principal, parent
POST	/api/behavior	Viselkedési bejegyzés rögzítése	teacher, homeroom_teacher, principal
DELETE	/api/behavior/[id]	Bejegyzés törlése	admin
GET	/api/communication/chat	Chat üzenetek lekérése	all
POST	/api/communication/chat	Chat üzenet küldése	all
DELETE	/api/communication/chat/[id]	Chat üzenet törlése	saját, admin
GET	/api/music	Zenekérések listája	all
POST	/api/music	Zenekérés beküldése	student, dj, anonymous
PUT	/api/music/[id]	Kérés státusz módosítása	dj
DELETE	/api/music/[id]	Zenekérés törlése	dj, admin
GET	/api/parent/children	Szülő gyermekeinek adatai	parent
POST	/api/parent/link	Gyermek összekapcsolása szülővel	admin
GET	/api/excuses	Igazolási kérelmek listája	homeroom_teacher, student, parent
POST	/api/excuses	Igazolás benyújtása	parent

Metódus	Végpont	Leírás	Jogosultság
PUT	/api/excuses/[id]	Igazolás jóváhagyása / elutasítása	homeroom_teacher
GET	/api/schedule-changes	Órarendváltások listája	all
POST	/api/schedule-changes	Órarendváltás rögzítése	admin
GET	/api/access	QR belépési napló lekérése	admin, principal
POST	/api/access	QR be-/kilépés rögzítése	hitelesített
POST	/api/upload/profile	Profilkép feltöltés (Cloudinary sign.)	hitelesített (saját)
GET	/api/classes	Osztályok listájának lekérése	all

3.5.3 HTTP státuszkódok

Kód	Jelentés	Mikor alkalmazzuk
200 OK	Sikeres	GET, PUT, DELETE sikeres végrehajtás
201 Created	Létrehozva	POST sikeres, új erőforrás létrehozva
400 Bad Request	Hibás kérés	Hiányzó vagy érvénytelen mezők
401 Unauthorized	Nem hitelesített	Hiányzó vagy lejárt JWT token
403 Forbidden	Tiltott	Nincs jogosultság az erőforráshoz
404 Not Found	Nem található	Az erőforrás nem létezik
500 Internal Error	Szerver hiba	Váratlan szerver oldali hiba

3.6 Biztonsági megvalósítás

3.6.1 Autentikáció és autorizáció

A rendszer Firebase Authentication JWT token alapú hitelesítést alkalmaz. Minden API végpont (kivéve /api/auth/*) ellenőrzi a Bearer token érvényességét a Firebase Admin SDK segítségével:

```
const authHeader = request.headers.get("Authorization")
if (!authHeader?.startsWith("Bearer ")) return 401
const token = authHeader.split("Bearer ")[1]
const decodedToken = await auth.verifyIdToken(token)
const userRole = decodedToken.role // Firebase Custom Claim
```

A JWT tokenek lejáratí ideje 1 óra, automatikus refresh-szel. A szerepkört Firebase Custom Claims tárolja - ezt az Admin SDK állítja be felhasználó létrehozásakor. A Firestore Security Rules adatbázis szinten is érvényesítik a jogosultságokat (firestore.rules), így a közvetlen adatbázis-hozzáférés is védett.

A szerepkör alapú hozzáférés-kezelés (RBAC) egy központi permissions.ts fájlban alapul:

```
export type UserRole = "admin" | "student" | "teacher" |
    "homeroom_teacher" | "parent" |
    "principal" | "dj"
```

3.6.2 RBAC mátrix

Az alábbi mátrix összefoglalja az egyes szerepkörök jogosultságait a főbb funkciókhoz:

Funkció	Admin	Igaz.	Osztályfőnök	Tanár	Diák	DJ	Szülő
Jegy rögzítése	N	N	I (magatartás)	I	N	N	N
Jegy megtekintése	I	N	I	I	I (saját)	I (saját)	I (gyermek)
Hiányzás rögzítése	N	N	I	I	N	N	N
Hiányzás megtekintése	N	N	I	I	I (saját)	I (saját)	I (gyermek)
Igazolás jóváhagyás	N	N	I	N	N	N	N
Házi feladat kiadás	N	N	I	I	N	N	N
Házi feladat beadás	N	N	N	N	I	I	N
Órarend szerkesztés	I	N	N	N	N	N	N
Viselkedés bejegyzés	N	I (igt.)	I (osztfőn., szaktanári)	I (szaktanári)	N	N	N
Felhasználó kezelés	I	N	N	N	N	N	N
Chat küldés	I	I	I	I	I	I	I
Zenékérés kezelés	I	N	N	N	N	I	N
Statisztikák	I	I	I (osztály)	I (saját)	N	N	N
QR kód generálás	N	N	N	N	I	I	N

3.6.3 Biztonsági intézkedések összefoglalója

Intézkedés	Megvalósítás	Véd ellene
Firebase Authentication	JWT token, 1 óra lejárta, auto-refresh	Jogosulatlan hozzáférés
Firestore Security Rules	Adatbázis szintű jogosultság-ellenőrzés	Direkt adatbázis hozzáférés
Input validáció	Szerver oldali validáció + sanitizálás	SQL injection, XSS
React XSS védelem	Automatikus HTML escape	Cross-site scripting
Env változók	.env fájl, nem verziókezel	API kulcs kiszivárgás
GDPR megfelelés	Adatkezelési irányelvek, törlési jog	Adatvédelmi szabálysértés

3.7 Tesztelés

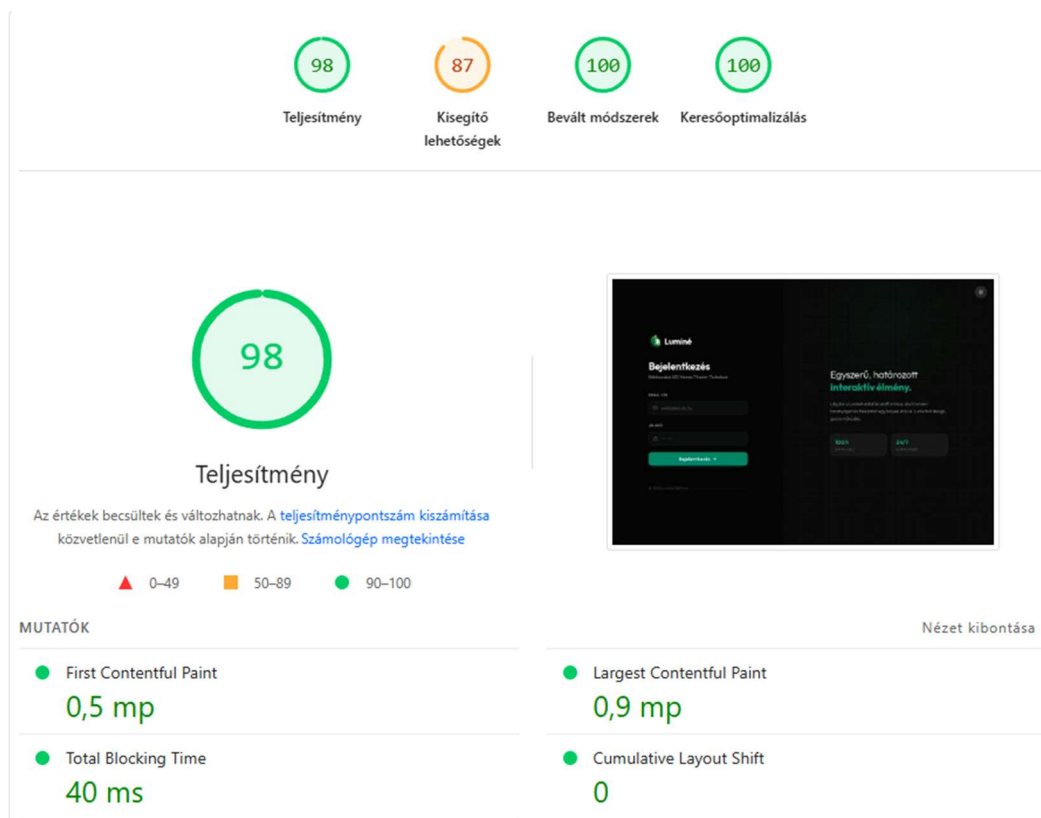
3.7.1 Tesztelési stratégia

A projekt háromszintű tesztelési stratégiát alkalmaz, amely biztosítja a kód minőségét és a rendszer megbízhatóságát:

- Unit tesztek (Vitest) egyedi komponensek és függvények tesztelése
- Integrációs tesztek API végpontok tesztelése
- End-to-End tesztek (Playwright) teljes felhasználói folyamatok tesztelése

3.7.2 Tesztelési eredmények

Tesztelési szint	Eszköz	Tesztek száma	Sikeres	Eredmény
Unit tesztek	Vitest	15	15	100%
E2E tesztek	Playwright	23	23	100%
Kód lefedettség	Vitest coverage	-	-	89.3%
Teljesítmény	Lighthouse	-	-	98/100
Akadálymentesség	Lighthouse	-	-	87/100
Bevált módszerek	Lighthouse	-	-	100/100
Keresőoptimalizálás	Lighthouse	-	-	100/100



3.7.3 Teljesítmény metrikák

Metrika	Érték	Értékelés
First Contentful Paint	0.5 s	Kiváló
Largest Contentful Paint	0.9 s	Kiváló
JavaScript bundle méret	2.1 MB (tömörítve: 580 KB)	Megfelelő
CSS méret	45 KB (tömörítve: 8 KB)	Kiváló

3.7.4 Főbb tesztesetek

Teszteset	Típus	Eredmény
Sikeres bejelentkezés admin fiókkal	E2E	PASS
Hibás jelszó 401 hibaüzenet megjelenik	E2E	PASS
Szerepkör szerinti átirányítás login után	E2E	PASS
Jegy rögzítése tanárként	E2E	PASS
Jegy megtekintése diákként (csak saját)	E2E	PASS
Jogosulatlan jegyrögzítés tiltása diákként	E2E	PASS
Hiányzás rögzítése tanárként	E2E	PASS
Igazolás benyújtása szülőként	E2E	PASS
Igazolás jóváhagyása osztályfőnökként	E2E	PASS
Házi feladat kiadása tanárként	E2E	PASS
Házi feladat beadása diákként	E2E	PASS
Chat üzenet küldése és megjelenítése	E2E	PASS
Zenekérés beküldése diákként	E2E	PASS
DJ elfogadja a zenekérést	E2E	PASS
QR kód generálás diákként	E2E	PASS
Profilkép feltöltés Cloudinary-ba	E2E	PASS
Admin: felhasználó létrehozása	E2E	PASS

Teszteteset	Típus	Eredmény
Órarend megtekintése tanárként	E2E	PASS
Viselkedési bejegyzés rögzítése	E2E	PASS
Szülő megtekinti a gyermek jegyeit	E2E	PASS
Kijelentkezés és session törlés	E2E	PASS
Reszponzív megjelenés mobil nézetben	E2E	PASS
Oldal betöltési idő (<3 s)	E2E	PASS
Átlagszámítás helyessége (súlyozás nélkül)	Unit	PASS
Átlagszámítás üres jegylistával	Unit	PASS
RBAC, tanár adhat jegyet	Unit	PASS
RBAC, diák nem adhat jegyet	Unit	PASS
Input validáció, érvénytelen email	Unit	PASS
Input validáció, hiányzó kötelező mező	Unit	PASS
QR kód generálás, egyedi token	Unit	PASS
Szerepkör felismerés JWT tokenből	Unit	PASS
Dátumformátum kezelés (ISO → HU)	Unit	PASS
Hiányzás százalék számítás	Unit	PASS
Cloudinary URL generálás	Unit	PASS
Firebase token érvényesítés (mock)	Unit	PASS
Órarend ütközés detektálás	Unit	PASS
Chat üzenet hossz validáció (max 500)	Unit	PASS
Viselkedési szint jogosultság ellenőrzés	Unit	PASS

3.8 Telepítés és üzemeltetés

3.8.1 Rendszerkövetelmények

Komponens	Követelmény
Node.js	18.0 vagy újabb
npm	9.0 vagy újabb
Git	Verziókezelőhöz
Firebase projekt	Firestore + Authentication engedélyezve

3.8.2 Telepítési lépések

1. Repository klónozása:

```
git clone https://git.gszi.edu.hu/vizsgaremek2526/vremek_13C_02.git
cd lumine
```

2. Függőségek telepítése:

```
npm install
```

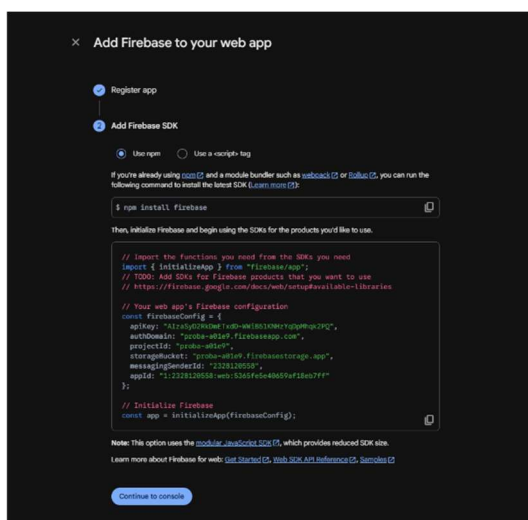
3. Környezeti változók beállítása:

#Szerkessze a .env fájlt Firebase és Cloudinary adataival **Vagy használja a mellékelt .env fájlt ahol található egy-egy teszt adatbázis**

#Firebase regisztrálás: [Ezen a linken!](#)

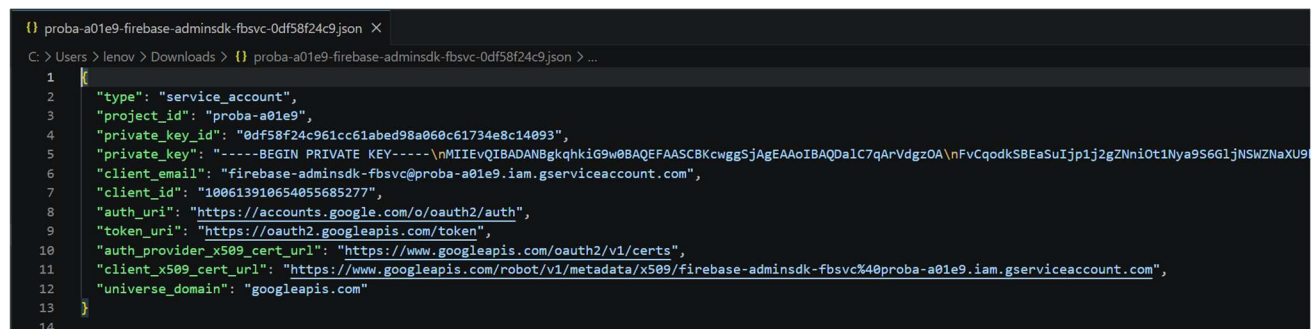
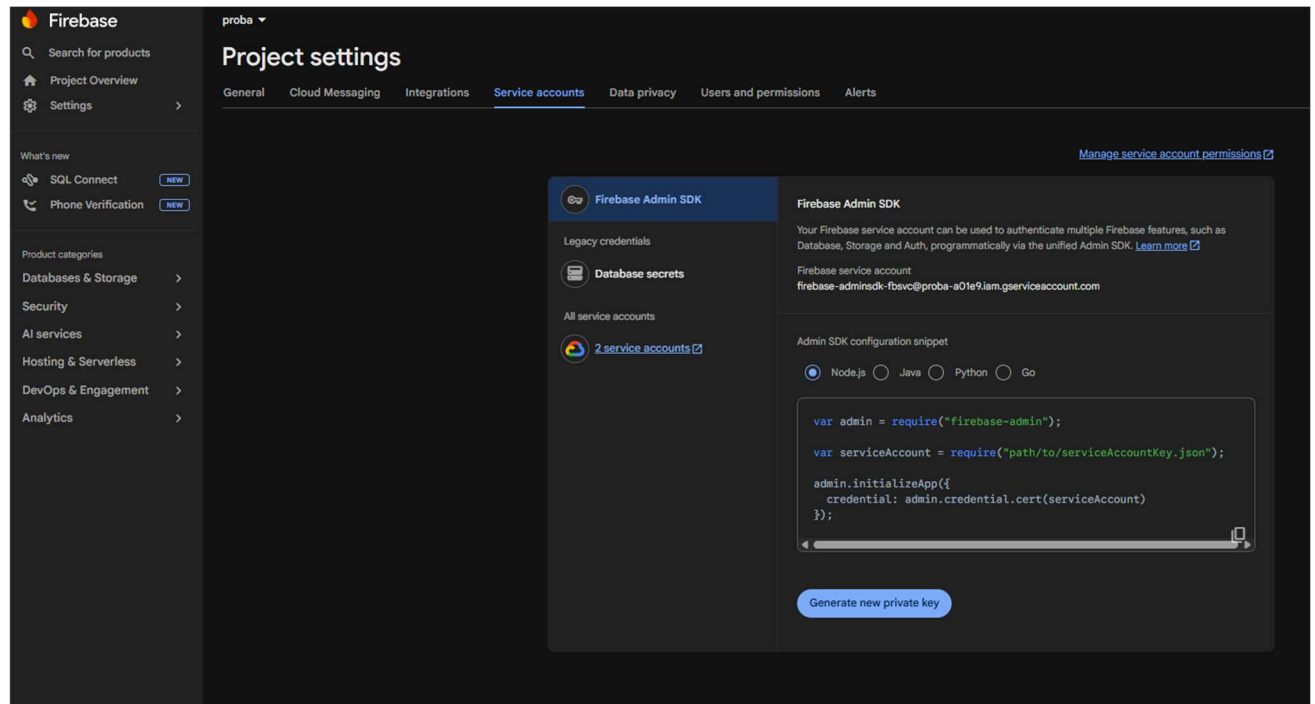
Hogyan töltsd ki(Firebase adatbázis):

1. Menj a Firebase Console-ra: <https://console.firebase.google.com>
2. Hozz létre egy új projektet, vagy nyiss meg egy meglévőt
3. Menj: Project Settings (fogaskerék ikon) > General > Your apps
4. Ha még nincs app, kattints a "</>" (Web) ikonra és regisztrálj egy új web appot
5. A megjelenő firebaseConfig objektumból másold ki az értékeket:



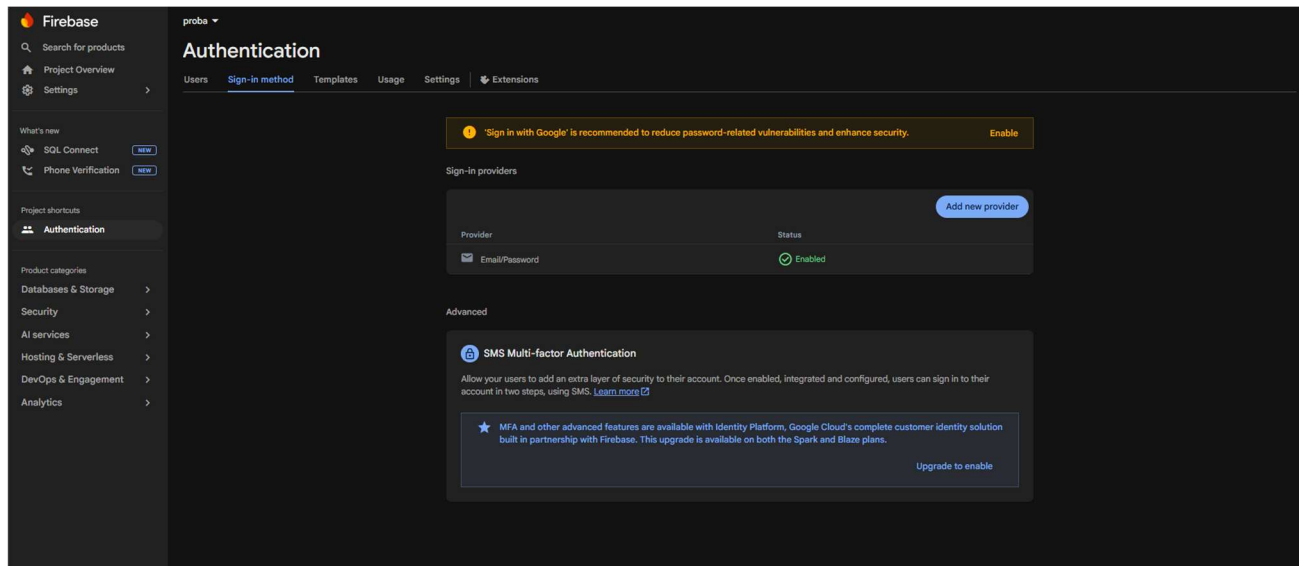
Hogyan töltsd ki (Firebase Admin SDK):

1. Menj: Project Settings > Service accounts
2. Kattints a "Generate new private key" gombra
3. A letöltött JSON fájlból másold ki:



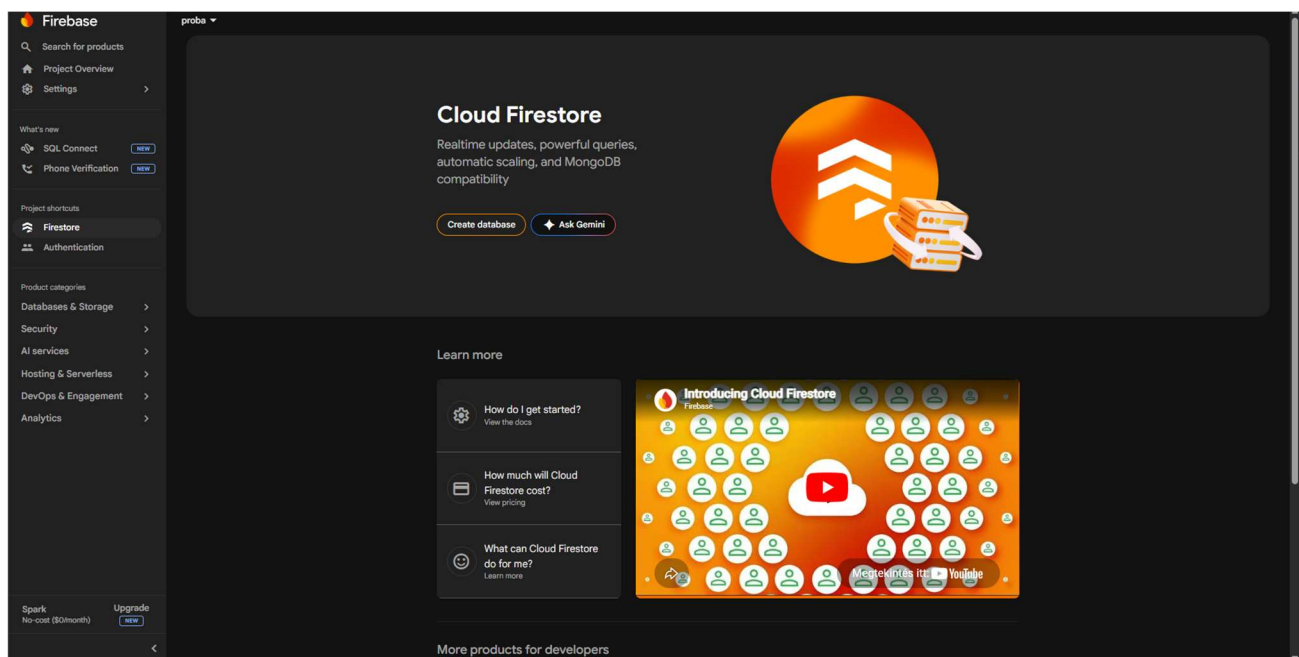
Authentication (bejelentkezéshez kell):

1. Menj: Build > Authentication > Get started
2. Sign-in method fülön keresd meg az "Email/Password" providert
3. Kattints rá és kapcsold be (Enable), majd mentsd el



Firestore (az adatok megjelenítéséhez kell, enélkül a táblák nem töltődnek be):

1. Menj: Databases & Storage > > Firestore > Create database
2. Válaszd a "Start in production mode"-ot, majd kattints Next
3. Válassz egy szerveret (pl. europe-west1), majd Done
4. A Rules fülön kattints az "Edit rules" gombra, és másold be a projektben lévő firestore.rules fájl tartalmát

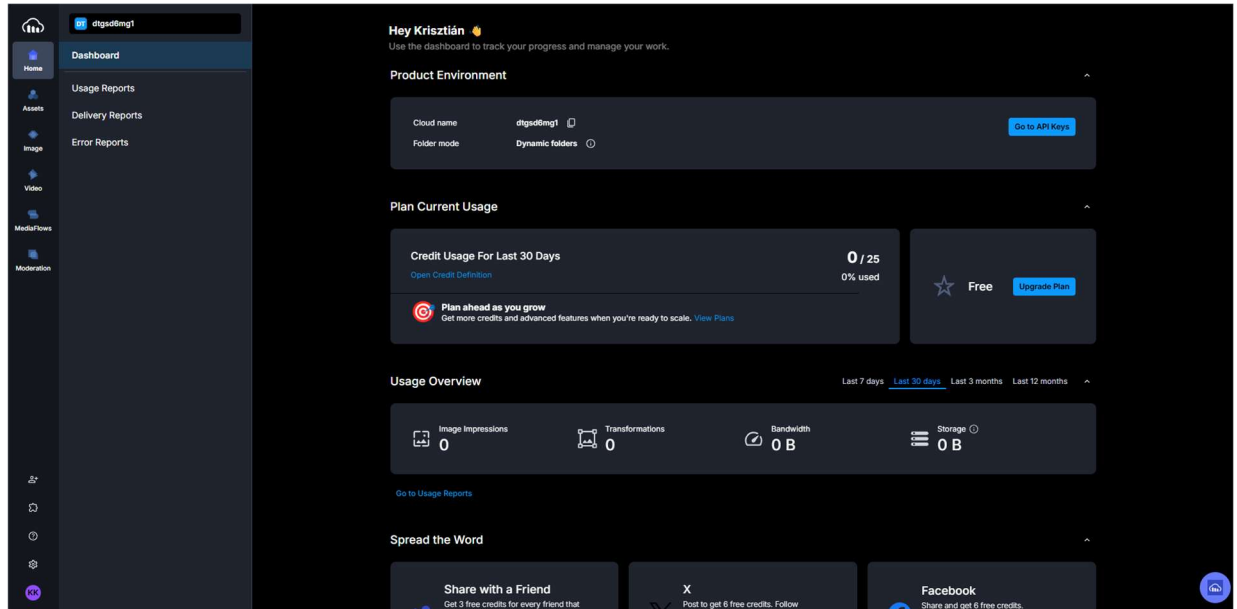


#Cloudinary regisztrálás: [Ezen a linken!](#)

Hogyan töltsd ki:

1. Regisztrálj vagy jelentkezz be: <https://cloudinary.com>
2. A Dashboard-on (<https://console.cloudinary.com>) találsz a "Cloud Name" értéket

NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME=pl. "dxy1abc23"



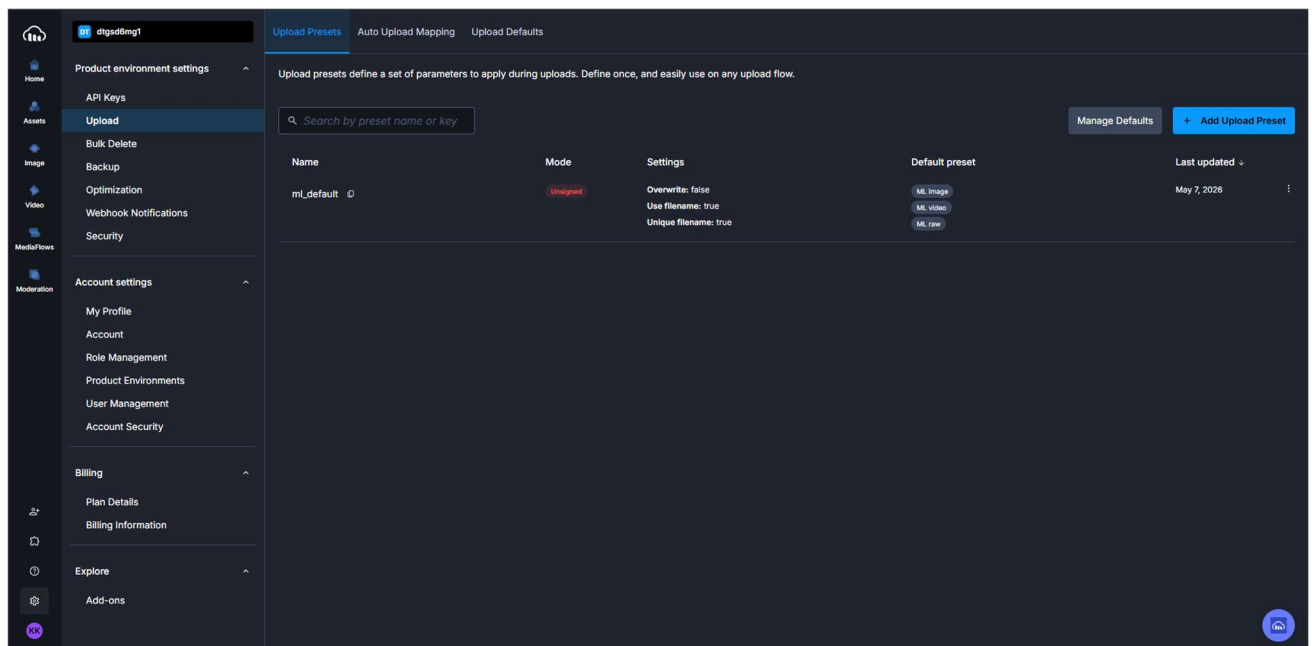
3. Upload Preset létrehozása:

Menj: Settings > Upload > Upload presets > Add upload preset

Signing Mode: "Unsigned" (hogy a frontendről is működjön)

Mentés után másold ki a preset nevét

NEXT_PUBLIC_CLOUDINARY_UPLOAD_PRESET=pl. "lumine_unsigned"



-

4. Fejlesztői szerver indítása:

```
npm run dev # http://localhost:3000
```

5. Teszt adatbázis inicializálása:

```
npm run db:init
```

!!Fontos!! Futnia kell a fejlesztői szervernek, anélkül nem képes inicializálni az adatokat

4. Felhasználói dokumentáció

4.1 Rendszerkövetelmények

Kategória	Részletek
Böngészők	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
Eszközök	Desktop (Win/Mac/Linux), Laptop, Tablet, Okostelefon
Internetkapcsolat	Minimum 1 Mbps letöltési sebesség
URL	https://lumine-project.vercel.app/

4.2 Bejelentkezés

A bejelentkezési folyamat lépései:

6. Nyissa meg a böngészőt és navigáljon a rendszer URL-jére
7. Adja meg az iskolai email címét (pl. `diak1@lumine.edu.hu`)
8. Adja meg a jelszavát (minimum 6 karakter)
9. Kattintson a "Bejelentkezés" gombra
10. A rendszer automatikusan átirányítja a szerepkörnek megfelelő dashboardra

Demo bejelentkezési adatok

Szerepkör	Email	Jelszó
Admin	<code>admin1@lumine.edu.hu</code>	<code>admin123456</code>
Tanár	<code>tanar1@lumine.edu.hu</code>	<code>tanar123456</code>
Diák	<code>diak1@lumine.edu.hu</code>	<code>diak123456</code>
Dj	<code>dj1@lumine.edu.hu</code>	<code>dj123456</code>
Osztályfőnök	<code>osztalyfonok1@lumine.edu.hu</code>	<code>osztalyfonok123456</code>
Szülő	<code>szulo1@lumine.edu.hu</code>	<code>szulo123456</code>
Igazgató	<code>igazgato1@lumine.edu.hu</code>	<code>igazgato123456</code>

4.3 Diák és Dj felhasználói útmutató

A diák és a dj dashboard a következő szekciókból áll:

Szekció	Tartalom
Dashboard	Mai órarend, legutóbbi jegyek, átlagok, közelgő házi feladatok
Órarend	Heti/napi nézet, navigálás hetente, óra részletek
Jegyek	Jegyek tantárgyanként, átlagok, grafikon
Mulasztások	Hiányzások listája, igazolás kérése
Házifeladat	Feladatok listája, beadás, visszajelzés megtekintése
Üzenőfal	Chat üzenetek, új üzenet küldése
Rádió	Zenekérés beküldése URL-lel, kérések státusza, emelet a dj tud törölni zenét
QR	Egyedi QR kód belépés/kilépés típussal
Profil	Személyes adatok, profilkép módosítása, statisztikák

4.4 Tanár felhasználói útmutató

Funkció	Lépések
Jegy rögzítése	Jegyek tab → Új jegy → Osztály, diák, tantárgy, jegy (1-5), cím → Mentés
Jelenlét rögzítése	Órarend tab → Óra kiválasztása → Jelenlét rögzítése → Diákok megjelölése → Mentés
Házi feladat kiadása	Házifeladat tab → Új házi feladat → Adatok, határidő → Kiadás
Beadás értékelése	Házifeladat → Kattintás → Beadások listája → Visszajelzés → Mentés
Viselkedési bejegyzés	Viselkedés tab → Új bejegyzés → Típus, szint, leírás → Mentés

4.5 Osztályfőnök felhasználói útmutató

Az osztályfőnök minden tanári funkcióhoz hozzáfér, valamint az alábbi kizárólagos funkciókhoz:

Funkció	Leírás
Igazolások kezelése	Igazolások tab → Kérelem → Jóváhagyás/Elutasítás + megjegyzés
Magatartás/szorgalom jegy	Jegyek tab → Magatartás/Szorgalom jegy → Diák, jegy, indoklás
Osztály áttekintés	Dashboard kártya - létszám, átlag

4.6 Szülő felhasználói útmutató

A szülő hozzáfér a gyermeke adataihoz, OM azonosítón keresztül, egyelőre nincs több gyermek hozzáadásra lehetőség de a program alkalmas változásra kevés ráfordított energiával:

- Órarend, Jegyek, Hiányzások, Házi feladatok, Viselkedési bejegyzések megtekintése
- Chat funkció - kommunikáció tanárokkal

4.7 Admin felhasználói útmutató

Modul	Funkciók
Felhasználók	Lista, létrehozás, szerkesztés, törlés, szerepkör kezelés
Órarend	Összes óra, létrehozás, szerkesztés, törlés
Jegyek	Összes jegy, szűrés, törlés
Statisztikák	Intézményi áttekintés, grafikonok

4.8 Igazgató felhasználói útmutató

Az igazgatói dashboard intézményi szintű áttekintést nyújt. A statisztikai adatok és az igazgatói szintű viselkedési bejegyzések kezelése az igazgató kizárólagos jogosultsága.

4.9 Hibaelhárítás - GYIK

Probléma	Megoldás
Nem tudok bejelentkezni	Ellenőrizze email/jelszó helyességét; törölje a cache-t; próbáljon inkognitó módban
Nem látom a jegyeket	Ellenőrizze a szűrőket (osztály, tantárgy); jelentkezzen ki és be újra
Lassú betöltés	Ellenőrizze internetkapcsolatot; zárja be a felesleges tabokat; frissítse az oldalt
Nem lehet beadni házi feladatot	Ellenőrizze, hogy nem járt-e le a határidő; próbálja újra
Elfelejtett jelszó	Vegye fel a kapcsolatot az iskola adminisztrátorával vagy IT részleggel
Offline funkciók	Megtekintés offline is elérhető; új adatok rögzítéséhez internet szükséges

5. Összegzés

5.1 Projekt eredményei

A Luminé iskolai adminisztrációs rendszer sikeres megvalósítása során egy modern, biztonságos és felhasználóbarát webalkalmazás jött létre, amely hatékonyan támogatja az oktatási intézmények mindennapi működését.

Eredmény kategória	Részletek
Funkcionális lefedettség	10 fő modul, 7 szerepkör, 30+ API végpont teljesen implementálva
Tesztelési eredmény	15/15 unit teszt (100%), 23/23 E2E teszt (100%)
Teljesítmény	Lighthouse 98/100, FCP 0.5s, LCP 0.9s
Biztonság	Firebase Auth, RBAC, GDPR megfelelés, Firestore Rules

5.2 Fejlesztési tapasztalatok

Pozitív tapasztalatok

- Next.js App Router egyszerűsített routing és API kezelés
- TypeScript a típusbiztonság már fejlesztés során kiszűrte a potenciális hibákat
- Firebase integráció zökkenőmentes auth + adatbázis, valós idejű szinkronizáció
- Tailwind CSS gyors, konzisztens UI fejlesztés
- Komponens újrafelhasználás csökkentette a kód duplikációt

Kihívások és megoldások

Kihívás	Megoldás
RBAC 7 szerepkörrel	Központosított permissions.ts, hasPermission() helper
Firestore korlátozott JOIN	Denormalizált adatstruktúra, composite indexek
Valós idejű chat	Firestore real-time listener, optimalizált frissítés
Mobil reszponzivitás	Mobile-first, bottom navigation, collapse szekciók
Firebase mock unit tesztekhez	Dependency injection, mock Firebase SDK

5.3 Jövőbeli fejlesztési lehetőségek

Időtáv	Fejlesztés	Prioritás
Rövid (3-6 hó)	Push értesítések (Firebase Cloud Messaging)	Magas
Rövid (3-6 hó)	Fájl feltöltés házi feladatokhoz (Firebase Storage)	Magas
Rövid (3-6 hó)	Jegyek/hiányzások exportálása (PDF, Excel)	Közepes
Középtávú (6-12 hó)	Mesterséges intelligencia tanulási nehézségek előrejelzése	Közepes
Középtávú (6-12 hó)	Mobil alkalmazás (React Native)	Magas
Hosszú (1-2 év)	Multi-tenant architektúra (több iskola)	Alacsony
Hosszú (1-2 év)	Automatikus tesztjavítás (AI)	Alacsony
Hosszú (1-2 év)	Gamifikáció, motivációs rendszer	Alacsony

6. Irodalomjegyzék

6.1 Könyvek és publikációk

1. Flanagan, D. (2020). JavaScript: The Definitive Guide, 7th Edition. O'Reilly Media.
2. Freeman, E., Robson, E. (2020). Head First Design Patterns, 2nd Edition. O'Reilly Media.
3. Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
4. Banks, A., Porcello, E. (2020). Learning React, 2nd Edition. O'Reilly Media.

Ezen a github linken értünk el még további könyveket
:<https://github.com/kpyszkowski/kpyszkowski>

Felhasznált referencia projekt:
: <https://github.com/zxmodren/Nextjs-SchoolManagementSystem-Template?tab=readme-ov-file>

6.2 Online dokumentációk

5. Next.js Documentation (2024). Next.js 14 App Router. <https://nextjs.org/docs>
6. React Documentation (2024). React 18. <https://react.dev>
7. TypeScript Documentation (2024). TypeScript Handbook. <https://www.typescriptlang.org/docs/>
8. Firebase Documentation (2024). <https://firebase.google.com/docs>
9. Tailwind CSS Documentation (2024). <https://tailwindcss.com/docs>
10. Vercel Documentation (2024). <https://vercel.com/docs>

6.3 Webes és technikai források

11. W3C (2024). WCAG 2.1 AA. <https://www.w3.org/WAI/WCAG21/quickref/>
12. Vitest (2024). <https://vitest.dev/>
13. Playwright (2024). <https://playwright.dev/>
14. Radix UI (2024). <https://www.radix-ui.com/>
15. GDPR (2018). General Data Protection Regulation. <https://gdpr.eu/>

LUMINÉ

Iskolai Adminisztrációs Rendszer • v1.0 • 2026

Békéscsabai SZC Nemes Tihamér Technikum és Kollégium

